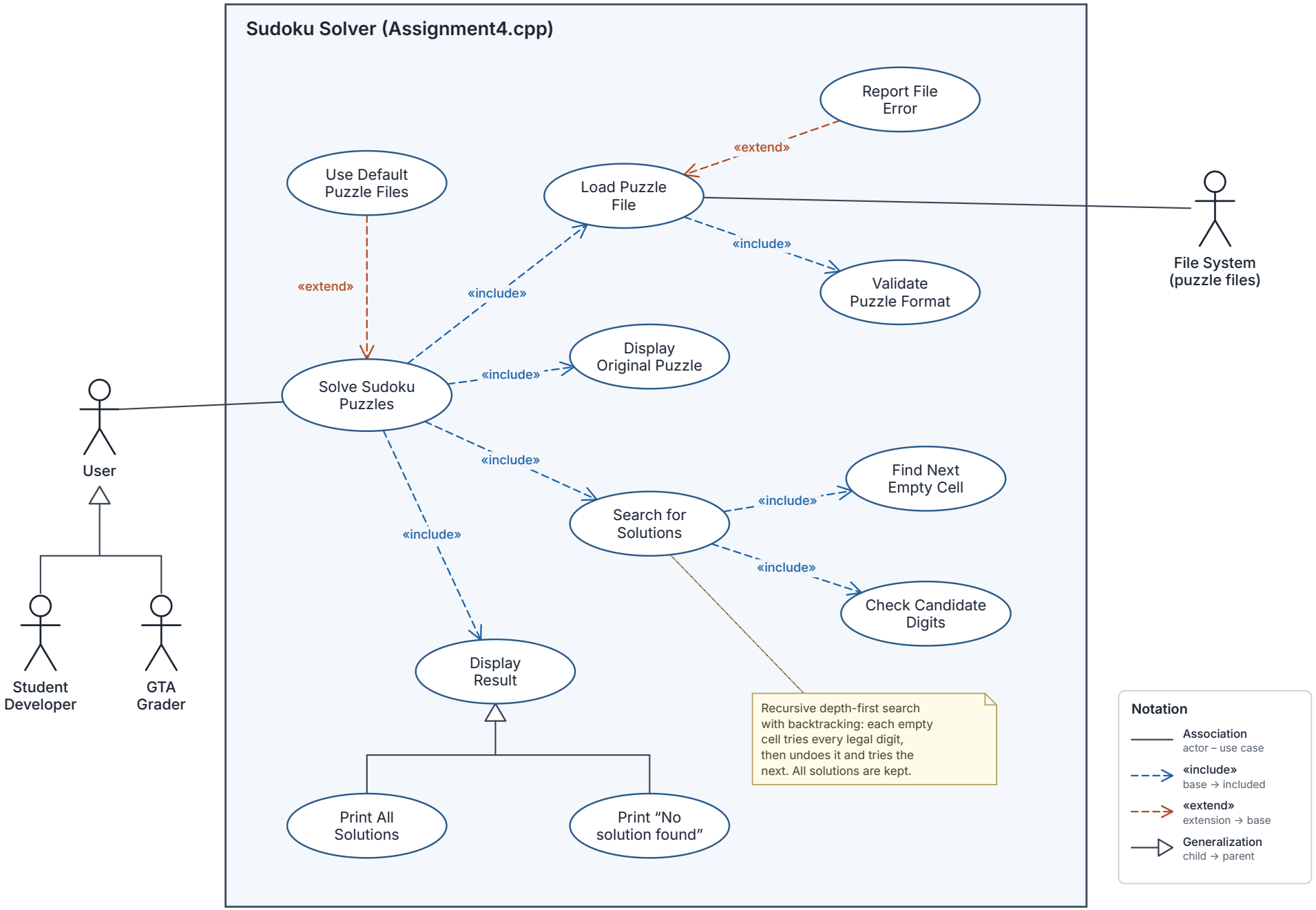

Sudoku Solver (Assignment4.cpp)
Report File Error
«extend»
Use Default Puzzle Files
Load Puzzle File
«include»
Validate Puzzle Format
File System (puzzle files)
«extend»
«include»
Display Original Puzzle
Solve Sudoku Puzzles
«include»
User
«include»
Find Next Empty Cell
Search for Solutions
«include»
«include»
Check Candidate Digits
Display Result
Recursive depth-first search with backtracking: each empty cell tries every legal digit, then undoes it and tries the next. All solutions are kept.
Print All Solutions
Print "No solution found"
Student Developer
GTA Grader
Notation
Association
actor – use case
«include»
base → included
«extend»
extension → base
Generalization
child → parent

Solve Sudoku Puzzles

The diagram on page 1 follows the four steps from the UML Use Case Modeling lecture. This section covers each step for the main use case, **Solve Sudoku Puzzles**.

Step 1. Identify the use cases

Use case	What it does
Solve Sudoku Puzzles	Main use case started by the User: run the program on one or more puzzle files.
Use Default Puzzle Files	«extend» If no file names are given, the program uses puzzle1.txt–puzzle5.txt.
Load Puzzle File	«include» Reads 81 entries from the file (digits 1–9, _ for blank) from the File System.
Validate Puzzle Format	«include» Checks each entry is a digit or underscore and that there are exactly 81.
Report File Error	«extend» If the file is missing or badly formatted, prints an error and moves on to the next file.
Display Original Puzzle	«include» Prints the file name and the puzzle as read.
Search for Solutions	«include» Recursive depth-first search with backtracking that finds every solution.
Find Next Empty Cell	«include» Chooses the empty cell to fill next (the one with the fewest legal digits).
Check Candidate Digits	«include» Tests whether a digit already appears in the same row, column, or 3×3 block.
Display Result	«include» Prints the outcome; specialized by the two use cases below.
Print All Solutions	Generalization of Display Result: prints every solution found, numbered if there is more than one.
Print "No solution found"	Generalization of Display Result: used when the search finds no solution.

Actors: the **User** (primary), specialized as Student Developer and GTA Grader; the **File System** (secondary), which supplies the puzzle files.

Step 2. Brief description

The User runs the Sudoku solver with one or more puzzle files (or none, to use puzzle1–5.txt). For each file the system loads and checks the puzzle, prints it, searches for every solution by recursive depth-first search with backtracking, and prints all solutions or "No solution found".

Step 3. Step-by-step flow (outline)

1. The User starts the program, optionally with puzzle file names.
2. For each puzzle file:
    a. **Include Load Puzzle File** and **Validate Puzzle Format**.
    b. **Include Display Original Puzzle**.
    c. **Include Search for Solutions**.
    d. **Include Display Result**.
3. The program ends after the last file.

A1 No file names given (step 1): **Use Default Puzzle Files**.

A2 File missing or invalid (step 2a): **Report File Error**, then skip to the next file.

A3 No solution exists (step 2d): Display Result is **Print "No solution found"**.

Step 4. Detailed description: Solve Sudoku Puzzles

Use case	Solve Sudoku Puzzles
Primary actor	User (Student Developer or GTA Grader)
Secondary actor	File System (provides puzzle1.txt–puzzle5.txt or other named files)
Goal	See every solution to each Sudoku puzzle, or confirm that a puzzle has none.
Trigger	The User runs `./Assignment4.exe` on an EECS Cycle server, with or without file names.
Preconditions	The program is compiled on the Cycle server. Each puzzle file is a 9×9 grid of digits 1–9 and underscores separated by spaces, in the same folder (or given by path).
Postconditions	For every file, the terminal shows the file name, the original puzzle, and either all solutions or "No solution found" (or an error message for an unreadable file). Puzzle files are not changed.
Relationships	Includes Load Puzzle File, Display Original Puzzle, Search for Solutions, and Display Result. Extended by Use Default Puzzle Files. Search for Solutions includes Find Next Empty Cell and Check Candidate Digits.

Main flow

1. The User runs the program with a list of puzzle file names.
2. The system takes the first file name from the list.
3. **Load Puzzle File:** the system opens the file from the File System and reads 81 entries, storing a digit as itself and an underscore as an empty cell.
4. **Validate Puzzle Format:** the system confirms every entry is valid and that there are exactly 81.
5. **Display Original Puzzle:** the system prints the file name and the puzzle, with _ for blanks.
6. The system checks that the given digits do not already repeat in any row, column, or block.
7. **Search for Solutions:**
   a. **Find Next Empty Cell** picks the empty cell with the fewest legal digits.
   b. **Check Candidate Digits** lists the digits 1–9 not already in that cell's row, column, or block.
   c. The system places the first candidate and repeats step 7 recursively for the next empty cell.
   d. When no empty cells remain (k = 81), the filled grid is saved as a solution.
   e. The system removes the digit (backtracks) and tries the next candidate, until every candidate at every level has been tried.
8. **Display Result** → **Print All Solutions:** the system prints each solution, numbered if there is more than one.
9. The system repeats steps 2–8 for each remaining file, then ends.

Alternate flows

A1 **No file names given** (step 1): **Use Default Puzzle Files** – the list becomes puzzle1.txt through puzzle5.txt.

A2 **File cannot be opened** (step 3): **Report File Error** – the system prints that the file could not be opened and continues at step 9.

A3 **Invalid entry or fewer than 81 entries** (step 4): **Report File Error** – the system names the problem and continues at step 9.

A4 **Given digits conflict** (step 6): the search is skipped and Display Result prints **"No solution found"**.

A5 **Search finds no solution** (step 7): a cell runs out of candidates on every path, so Display Result prints **"No solution found"** (for example, puzzle4.txt).

Special requirements

1. The search must be brute force (depth-first search with backtracking), not logical deduction.
2. All solutions must be printed when a puzzle has more than one (for example, puzzle5.txt has two).
3. The program must be object-oriented C++ in a single file, Assignment4.cpp, and must run correctly on the EECS Cycle servers.
4. Each puzzle should finish in well under a second.